Федеральное агентство железнодорожного транспорта

Омский государственный университет путей сообщения

Кафедра «Автоматика и системы управления»

Допущен к защите
«____» _____ 2023 г.

_____ _____
Подпись Расшифровка подписи

ПРОГРАММИРОВАНИЕ ЧИСЛЕННЫХ МЕТОДОВ
Пояснительная записка к курсовой работе по дисциплине
«Теоретические основы аппаратно-программных средств»
ИНМВ.401100.000 ПЗ

Оценка
«____» _____ 2023 г.

_____ _____
Подпись Расшифровка подписи

Студент гр. 22 м

_____ Е.А.Кучеренко
«__»_____ 2023 г.

Доцент кафедры АиСУ
_____ А.С. Окишев
«__»_____ 2023 г.

Омск 2023

Реферат

УДК 519.6

Пояснительная записка содержит 46 страниц, 17 рисунков, 2 таблицы, 4 источника.

Численный метод, нелинейное уравнение, корень, итерация, сходимость, аппроксимация, интерполяция, задача Коши, обыкновенное дифференциальное уравнение.

Объектом исследования являются приближенные численные методы решения некоторых математических и инженерных задач, а также программное обеспечение, реализующее эти методы.

Цель работы – ознакомиться с численными методами решения нелинейных и дифференциальных уравнений и интерполяции функций, решить предложенные типовые задачи с помощью предоставленного преподавателем программного обеспечения, сформулировать выводы по полученным решениям, отметить достоинства и недостатки методов, сравнить удобство использования и эффективность работы каждой программы.

Пояснительная записка к курсовой работе оформлена в текстовом редакторе Microsoft Office 2010. Графики нелинейных функций построены с помощью программы Visual Studio Code. При решении обыкновенных дифференциальных уравнений использовалась среда математического моделирования Python.

Содержание

Введение 4
1 Решение нелинейных уравнений 5
1.1 Метод простых итераций 6
1.2 Метод Ньютона 8
1.3 Решение нелинейного уравнения методом простых итераций 10
1.4 Решение нелинейного уравнения методом Ньютона 12
1.5 Вывод 14
2 Интерполяция функций 15
2.1 Локальная и глобальная интерполяция 16
2.2 Кусочно-линейная интерполяция 16
2.3 Кусочно-квадратичная интерполяция 17
2.4 Кубические сплайны 17
2.5 Многочлен Лагранжа 19
2.6 Кусочно-квадратичная интерполяция функции по таблице значений 20
2.7 Интерполяция функции кубическим сплайном по таблице значений 24
2.8 Интерполяция функции многочленом Лагранжа по таблице значений 28
2.9 Вывод 34
3 Решение обыкновенных дифференциальных уравнений 35
3.1 Метод Эйлера 35
3.2 Метод Эйлера-Коши 37
3.3 Решение ОДУ методом Эйлера 37
3.4 Решение ОДУ методом Эйлера-Коши 40
3.5 Вывод 44
Заключение 45
Библиографический список 46

Введение

В связи с развитием новой вычислительной техники инженерная практика наших дней все чаще и чаще встречается с математическими задачами, точное решение которых получить весьма сложно или невозможно. В этих случаях обычно прибегают к тем или иным приближенным вычислениям. Вот почему приближенные и численные методы математического анализа получили за последние годы широкое развитие и приобрели исключительно важное значение.

Новые вычислительные средства вызвали переоценку известных методов решения задач с точки зрения целесообразности их реализации на ЭВМ и стимулировали создание более эффективных, что привело к появлению новой дисциплины – вычислительной математики. Предметом изучения последней являются численные методы решения задач математического анализа: изучение алгоритмов и условий сходимости итерационных методов, определение границ применимости методов, исследования оценок погрешностей методов и вычислений. Главным разделом вычислительной математики является реализация численных методов на ЭВМ, то есть составление программы для требуемого алгоритма и решения с ее помощью конкретной задачи.

Любая прикладная задача формируется исходя из определенного физического смысла некоторого процесса (распределение тепла в стержне, описание траектории движения объектов). Прикладная математическая задача может быть сформулирована, например, из описания некоторой экономической модели (задача распределения ресурсов, задача планирования производства, транспортная задача перевозки грузов, оптимальных в заданном смысле). Следовательно, для постановки любой прикладной задачи нужна математическая модель. Поэтому, можно выделить следующие этапы решения задач на ЭВМ:

1) описание математической модели задачи на основе физической или экономической модели;

2) изучение методов решения поставленной математической модели задачи и создание новых методов;

3) выбор метода решения задачи исходя из заданной точности решения и особенностей задачи;

4) составление блок-схемы программы для решения задачи на ЭВМ;

5) отладка программы и оценка полученных результатов;

6) решение задачи на ЭВМ, построение графиков, получение оценки погрешностей, обоснование результатов.

В курсовой работе рассматриваются не прикладные, а типовые математические задачи, которые могут возникнуть при переходе от реальных систем к их математическим моделям, поэтому основное внимание уделяется последнему этапу.

1 Решение нелинейных уравнений

Нелинейными уравнениями называются уравнения вида

$$f(x)=0, \tag{1.1}$$

где $f(x)$ – нелинейная функция, которая может относиться к трем типам:

1) нелинейная алгебраическая функция вида

$$a_n x^n + a_{n-1} x^{n-1} + ... + a_1 x + a_0;$$

2) трансцендентные функции – тригонометрические, обратные тригонометрические, логарифмические, показательные и гиперболические функции;

3) различные комбинации этих функций, например, $f(x)=x^2+\sin x$.

Решением нелинейного уравнения (1.1) является такая точка x^*, которая при подстановке в уравнение (1.1) обращает его в тождество. На практике не всегда удается подобрать такое решение. В этом случае решение уравнения (1.1) находят с применением приближенных (численных) методов. Тогда решением будет являться такая точка x^*, при подстановке которой в уравнение (1.1) последнее будет выполняться с определенной степенью точности, т.е. $\left|f(x^*)\right| \le \varepsilon$, где ε – малая величина. Нахождение таких решений и составляет основу численных методов и вычислительной математики.

Решение нелинейных уравнений разделяется на два этапа: отделение корней уравнений и уточнение корней нелинейных уравнений.

На первом этапе необходимо исследовать уравнение и выяснить, имеются корни или нет. Если корни имеются, то узнать, сколько их, и затем определить интервалы, в каждом из которых находится единственный корень.

Первый способ отделения корней – графический. Исходя из уравнения (1.1), можно построить график функции $y=f(x)$. Тогда точка пересечения графика с осью абсцисс является приближенным значением корня. Если *f*(*x*) имеет сложный вид, то ее можно представить в виде разности двух функций $f(x)=f_1(x)-f_2(x)$. Так как $f(x)=0$, то выполняется равенство $f_1(x)=f_2(x)$. Если построить два графика $y_1=f_1(x)$, $y_2=f_2(x)$, то абсцисса точки их пересечения будет приближенным значением корня уравнения.

Второй способ отделения корней нелинейных уравнений – аналитический. Он основывается на следующих трех теоремах.

Теорема 1. Если функция $f(x)$ непрерывна на отрезке $[a;b]$ и меняет на концах отрезка знак (т.е. $f(a)\cdot f(b)<0$), то на $[a;b]$ содержится хотя бы один корень.

Теорема 2. Если функция $f(x)$ непрерывна на отрезке $[a;b]$, выполняется условие вида $f(a)\cdot f(b)<0$ и производная $f'(x)$ сохраняет знак на $[a;b]$, то на отрезке имеется единственный корень.

Теорема 3. Если функция $f(x)$ является многочленом *n*-ой степени и на концах отрезка $[a;b]$ меняет знак, то на $[a;b]$ имеется нечетное количество корней (если производная $f'(x)$ сохраняет знак на $[a;b]$, то корень единственный). Если на концах отрезка $[a;b]$ функция не меняет знак, то уравнение (1.1) либо не имеет корней на $[a;b]$, либо имеет четное количество корней.

При аналитическом методе исследований необходимо выявить интервалы монотонности функции $f(x)$. Для этого необходимо вычислить критические точки $(\xi_1, \xi_2, ..., \xi_n)$, в которых первая производная $f'(\xi_i)$ равна нулю или не существует. Тогда вся числовая ось разбивается на интервалы монотонности (ξ_i, ξ_{i+1}). На каждом из них определяется знак производной $f'(x_i)$, где $x_i \in (\xi_i, \xi_{i+1})$. Затем выделяются те интервалы монотонности, на которых функция $f(x)$ меняет знак.

На втором этапе на каждом из этих интервалов для поиска корня используются численные итерационные методы уточнения корней, например методы половинного деления, простых итераций или Ньютона.

1.1 Метод простых итераций

Пусть известно, что нелинейное уравнение (1.1) имеет на отрезке $[a,b]$ единственный вещественный корень $x^* \in [a,b]$. Требуется найти этот корень с заданной точностью. Применяя тождественные преобразования, приведем уравнение к виду

$$x = \varphi(x). \tag{1.2}$$

Выберем произвольно приближенное значение корня $x_0 \in [a,b]$ и вычислим $\varphi(x_0) = x_1$. Найденное значение x_1 подставим в правую часть соотношения (1.2) и вычислим $\varphi(x_1) = x_2$. Продолжая процесс вычислений дальше, получим числовую последовательность $x_0, x_1, x_2, ...$ Если существует предел этой последовательности, то он и является корнем уравнения (1.2). В самом деле, пусть $\lim\limits_{k\to\infty} x_k = x^*$. Тогда, переходя к пределу в равенстве (1.2)

$$\lim_{k\to\infty} x_{k+1} = \lim_{k\to\infty} \varphi(x_k)$$

и учитывая непрерывность функции $\varphi(x_k)$ на отрезке $[a,b]$, получим

$$\lim_{k\to\infty} x_{k+1} = \varphi\left(\lim_{k\to\infty} x_k\right) \text{ или } x^* = \varphi(x^*).$$

Корень можно вычислить с заданной точностью по итерационной формуле

$$x_{k+1} = \varphi(x_k), \quad k = 0, 1, 2, ... \tag{1.3}$$

Достаточное условие, при котором итерационный процесс сходится, определяет следующая теорема: пусть функция $\varphi(x)$ определена и дифференцируема на отрезке $[a,b]$, причем все ее значения $\varphi(x)\in[a,b]$ и выполняется условие

$$|\varphi'(x)|\le q<1, \quad \forall x\in[a,b]. \tag{1.4}$$

тогда процесс итераций (1.3) сходится независимо от начального значения $x_0\in[a,b]$ и предельное значение $x^*=\lim\limits_{k\to\infty}x_k$ является единственным корнем уравнения (1.2) на $[a,b]$.

Геометрическая интерпретация метода простых итераций заключается в следующем: если построить два графика функций $y=x$ и $y=\varphi(x)$, то абсцисса точки их пересечения будет корнем x^*. Построим итерационный процесс. Зададим $x_0\in[a,b]$. Вычислим $x_1=\varphi(x_0)$ – первое приближение и $x_2=\varphi(x_1)$ – второе приближение. В первом случае (рисунок 1.1, а) процесс сходящийся ($|\varphi'(x)|<1$), во втором (рисунок 1.1, б) – расходящийся ($|\varphi'(x)|>1$).

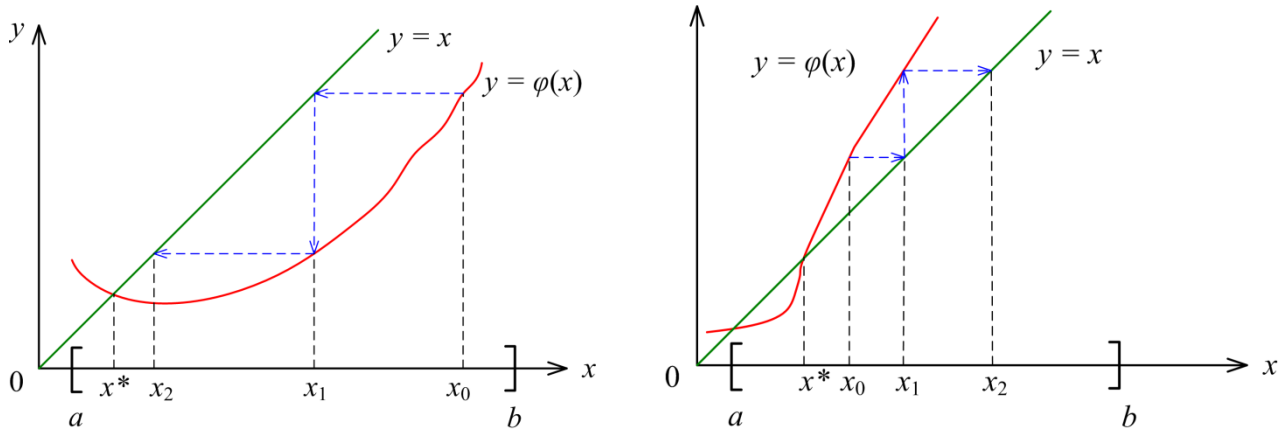


Рисунок 1.1 – Сходящийся (а) и расходящийся (б) итерационные процессы

Часто, если итерационный процесс расходится из-за невыполнения условия (1.4), нелинейное уравнение можно привести к виду, допускающему сходящиеся итерации.

Выполнения условия сходимости можно добиться путем перехода от исходного уравнения $f(x)=0$ к эквивалентному виду $x=\varphi(x)$ следующим образом: сначала умножить обе части уравнения (1.1) на $c=\text{const}\neq 0$, а затем прибавить к обеим частям x, тогда $x+cf(x)=x$. Обозначив $\varphi(x)=x+cf(x)$, получим уравнение (1.2). Константа c выбирается так, чтобы выполнялось достаточное условие сходимости итерационного процесса (1.4), т.е.

$$|\varphi'(x)|=|1+cf'(x)|<1, \quad \forall x\in[a,b]. \tag{1.5}$$

Условие (1.5) равносильно двойному неравенству

$$-1<1+cf'(x)<1,$$

поэтому константа выбирается из соотношений:

$$\begin{cases} \dfrac{-2}{f'(x)}<c<0, \quad f'(x)>0, \forall x\in[a,b]; \\ 0<c<\dfrac{-2}{f'(x)}, \quad f'(x)<0, \forall x\in[a,b]. \end{cases}$$

Метод простых итераций и почти все другие итерационные методы имеют два достоинства:

1) являются универсальными и самоисправляющимися, то есть любая неточность на каком-либо шаге итераций отражается не на конечном результате, а лишь на количестве итераций. Подобные ошибки устойчивы даже по отношению к грубым ошибкам (сбоям ЭВМ), если только ошибка не выбрасывает очередное приближение за пределы области сходимости;

2) позволяют достигнуть любой заданной точности при любом начальном приближении $x_0\in[a,b]$.

Недостатки методов:

– трудность приведения уравнения (1.1) к виду (1.2).

– если начальное приближение x_0 находится далеко от корня, то число итераций при этом увеличивается, а объем вычислений возрастает.

Процесс итераций заканчивается при выполнении двух критериев:

1) Когда два последних приближения отличается между собой по модулю на заданную величину ε:

$$|x_{k+1}-x_k|\le\varepsilon. \tag{1.6}$$

Одного критерия (1.6) недостаточно, так как в случае крутизны графика, данное условие будет выполнено, но x_{k+1} может находиться далеко от корня;

2) Когда последнее вычисленное приближение к корню удовлетворяет уравнению с заданной точностью:

$$|f(x_{k+1})|\le\delta. \tag{1.7}$$

Отдельно критерия (1.7) бывает недостаточно, так как при пологой функции $f(x)$ условие может быть выполнено, но x_{k+1} может быть далеко от корня.

1.2 Метод Ньютона

Пусть уравнение (1.1) имеет на интервале $[a,b]$ единственный корень, причем существует непрерывная на $[a,b]$ производная $f'(x)$. Метод Ньютона служит для уточнения корней нелинейных уравнений в заданном интервале. Его можно рассматривать как частный случай метода простых итераций, если

$$\varphi(x)=-\frac{f(x)}{f'(x)}+x.$$

Тогда итерационный процесс осуществляется по формуле:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad k = 0, 1, \ldots. \tag{1.8}$$

Геометрически этот процесс представлен на рисунке 1.2. Он означает замену на каждой итерации k графика кривой $y = f(x)$ касательной к ней в точках с координатами (x_k, y_k).

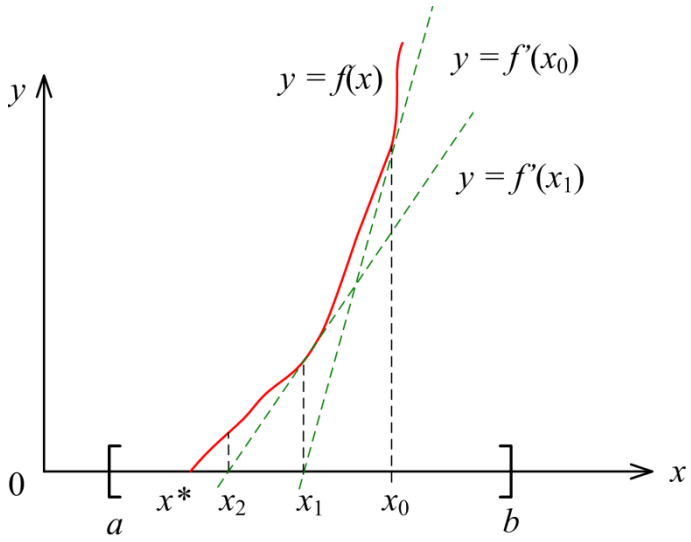


Рисунок 1.2 – Геометрическая интерпретация метода Ньютона

Достаточное условие сходимости обеспечивается выбором начальной точки x_0. Начальным приближением x_0 служит один из концов отрезка $[a,b]$, в зависимости от того, в каком из них выполняется достаточное условие сходимости

$$f(x_0) f''(x_0) > 0. \tag{1.9}$$

При произвольном начальном приближении итерации сходятся, если

$$|\varphi'(x)| = \left| \frac{f(x) \cdot f''(x)}{(f'(x))^2} \right| < 1 \quad \forall x \in [a,b]. \tag{1.10}$$

Метод Ньютона рекомендуется применять для нахождения простых действительных корней уравнения (1.1).

Достоинством метода является то, что он обладает скоростью сходимости, близкой к квадратичной.

Недостатки метода:

– не при любом начальном приближении метод Ньютона сходится, а лишь при таком, для которого $f'(x_0) \neq 0$;

– если $f'(x_k) \to 0$, т.е. касательная к графику почти параллельна оси абсцисс, то $x_{k+1} - x_k \to \infty$ и метод расходится;

– если $f'(x_n) \to \infty$, т.е. касательная к графику почти параллельна оси ординат, то $x_{k+1} - x_k \to 0$ и продвижения к корню не будет.

Последних трудностей можно избежать, применив модификацию метода Ньютона, в которой используется только касательная в точке начального приближения. Рабочая формула при этом имеет вид:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_0)}. \qquad (1.11)$$

1.3 Решение нелинейного уравнения методом простых итераций

Задание: найти приближенное численное решение нелинейного уравнения методом простых итераций с точностью $\varepsilon = 10^{-3}$:

$$2x + \lg x = 0. \qquad (1.12)$$

Для приблизительной оценки корней нелинейного уравнения (1.12) построим график функции с помощью программы Visual Studio Code (рисунок 1.3).

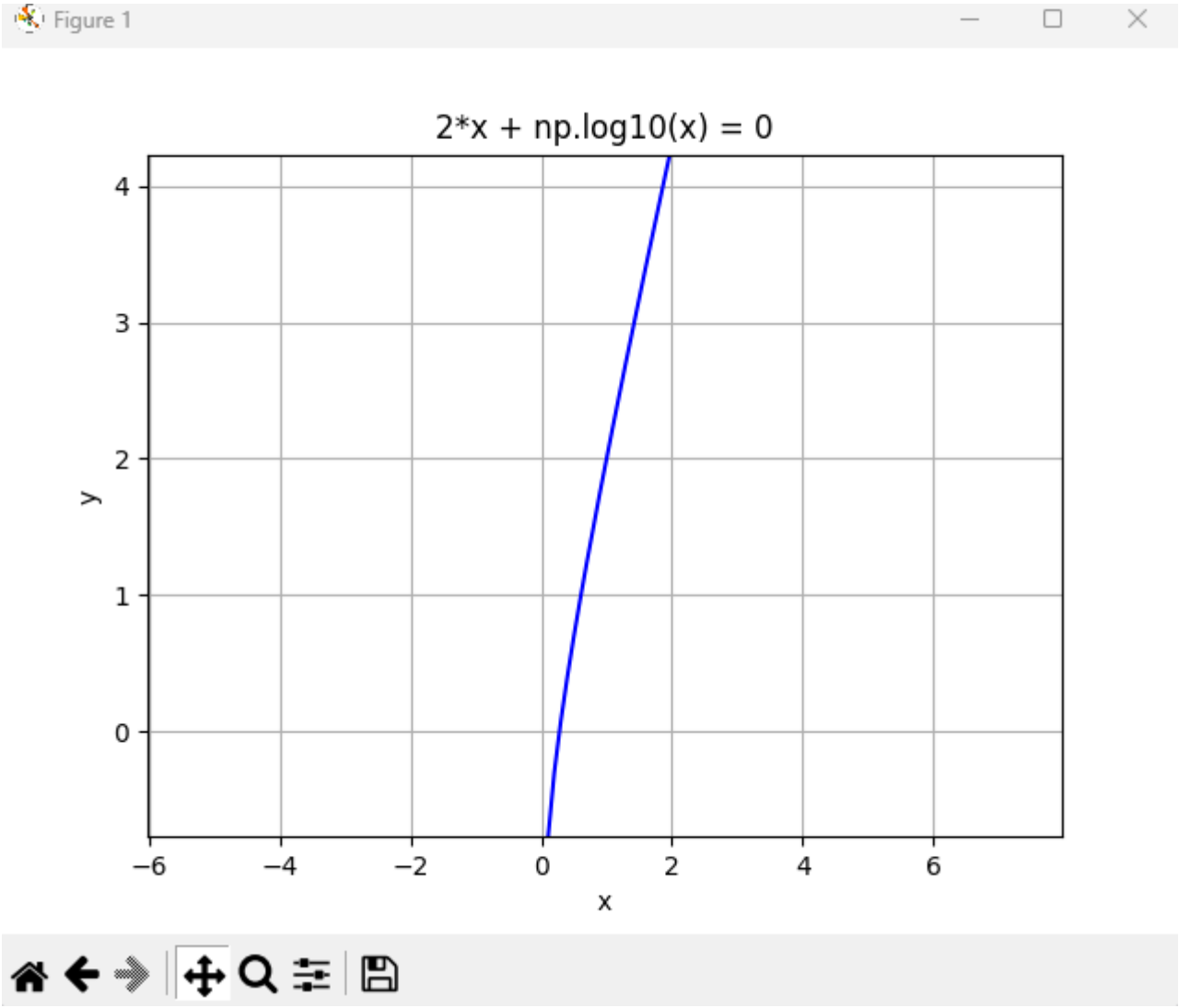


Рисунок 1.3 – График нелинейной функции y = 2^{-x} + 1 – 0,15 x = 0

Из рисунка 1.3 видно, что график функции пересекает ось абсцисс в точке, расположенной на отрезке [0; 2]. В качестве начального приближения к корню выбираем точку 1.

С помощью программы Python напишем код для решения нелинейного уравнения методом простых итераций.

Основной скрипт, в котором вводятся исходные данные, выполняется построение графика для отделения корней и выводится результат (листинг 1.1);

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(-10, 10, 0.1)
plt.figure(1)
y = 2*x+np.log10(x)
plt.plot(x, y, 'b')
plt.axis([-6, 8, -1, 4])
plt.grid(True)
plt.xlabel('x')
plt.ylabel('y')
plt.title('2*x + np.log10(x) = 0')
plt.show()

epsilon = 0.001
x0 = 1
k = 0
x = x0
kmax = 100
D = 50

while abs(D) > epsilon and k < kmax:
    y = 2*x+np.log10(x)
    y = y/-3
    xn = x + y
    D = abs(xn - x) # Погрешность
    x = xn
    k = k + 1

print('Корень:')
print(format(x, '.4f'))
print('Количество итераций:')
print(k)
print('Погрешность:')
print(format(D, '.4f'))
```

Листинг 1.1 – Вычисление значения функции, лист 1

Результат работы программы представлен на рисунке 1.4.

```
Корень:
0.2780
Количество итераций:
5
Погрешность:
0.0003
```

Рисунок 1.4 – Результат работы программы

В данном случае итерационный процесс сходится; корень уравнения x = 0.2780 найден с заданной точностью $\varepsilon = 0,001$ за 5 итераций, погрешность решения составляет 0,0003.

1.4 Решение нелинейного уравнения методом Ньютона

Задание: найти приближенное численное решение нелинейного уравнения методом Ньютона с точностью $\varepsilon = 10^{-3}$:

$$2^{-x} + 1 - 0,15\,x = 0. \qquad (1.13)$$

Для приблизительной оценки корней нелинейного уравнения (1.13) построим график функции с помощью программы Visual Studio Code (рисунок 1.5).

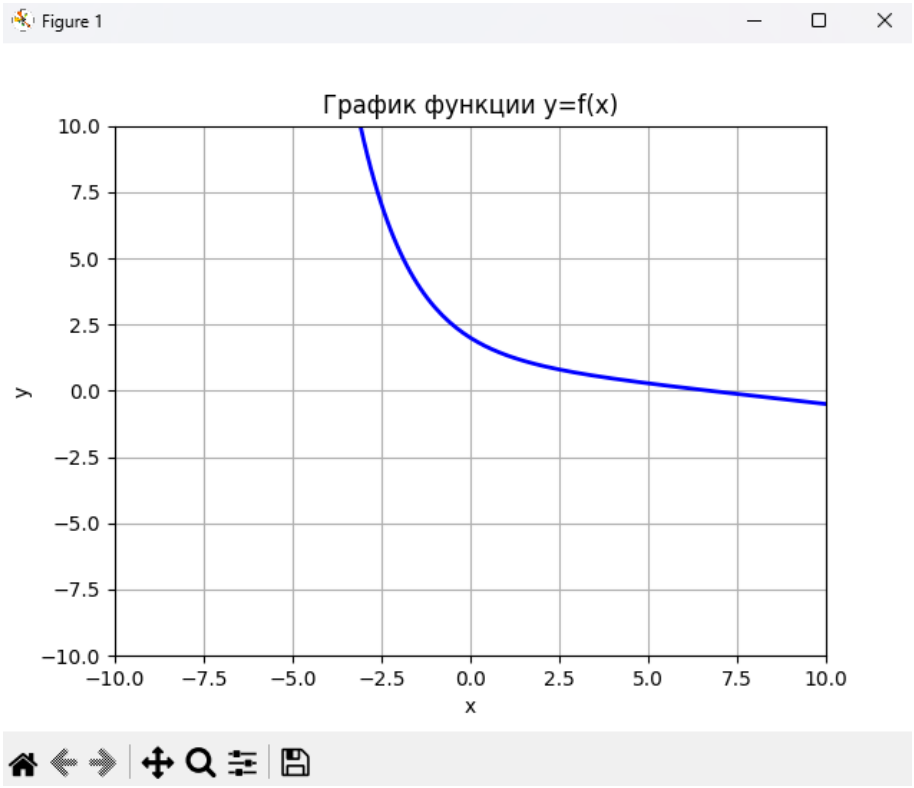


Рисунок 1.5 – График нелинейной функции $y = f(x)$

График функции пересекает ось абсцисс в точке, находящейся на отрезке [5; 7,5]. В качестве начального приближения к корню выбираем точку 6

С помощью программы Python напишем код для решения нелинейного уравнения методом Ньютона (листинг 1.5).

```
import numpy as np
import matplotlib.pyplot as plt

#  Этап 1: Отделение корней уравнения
x = np.arange(-10, 10.1, 0.1) # изменение x от -10 до 10 с шагом
0.1
y = 2**(-x) + 1 - 0.15*x # функция
plt.figure(1)
plt.plot(x, y, '-b', linewidth=2) # построение графика
```

Листинг 1.2 – Решение нелинейного уравнения методом Ньютона, лист 1

```
plt.axis([-10, 10, -10, 10])
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True) # включить сетку графика
plt.title('График функции y=f(x)')
plt.show() # демонстрация

# Этап 2: Уточнение корней уравнения
def Newton(x, F, F1): # Метод Ньютона
    D = -F / F1
    x1 = x + D
    return (x1, D)

def fun1(x):
    F = 2**(-x) + 1 - 0.15*x
    F1 = -(np.log(2) / 2**x) - 3/20
    return (F, F1)

eps = 0.001 # допустимая погрешность
kmax = 50 # максимальное число итераций
x = 6 # начальное приближение
k = 0 # начальное число итераций
F, F1 = fun1(x) # нахождение первой производной
x1, D = Newton(x, F, F1)
k = k + 1
print('%i\t %f\t %f' % (k, x1, D))

while (abs(D) > eps) and (k < kmax):
    x = x1
    F, F1 = fun1(x)
    x1, D = Newton(x, F, F1)
    k = k + 1
    print('%i\t %f\t %f' % (k, x1, D))
print('Корень: x = %f' % x1)
print('Количество итераций: k = %i' % k)
print('Погрешность: D = %f' % D)
```

Листинг 1.2, лист 2

Результат работы программы представлен на рисунке 1.6.

```
1       6.718925        0.718925
2       6.729490        0.010565
3       6.729491        0.000002
Корень: x = 6.729491
Количество итераций: k = 3
Погрешность: D = 0.000002
```

Рисунок 1.6 – Результат работы программы

Корень уравнения x = 6.729 найден с заданной точностью $\varepsilon = 0{,}001$ за 3 итерации, погрешность решения составляет 0.000002.

1.5 Вывод

Рассмотрены два метода решения нелинейных уравнений: Ньютона и простых итераций.

Метод простых итераций является универсальным и самоисправляющимся, то есть любая неточность на каком-либо шаге итераций отражается не на конечном результате, а лишь на количестве итераций. Он основан на построении сходящейся итерационной последовательности и использует только значения самой функции, поэтому обладает линейной сходимостью. Недостаток метода простых итераций в том, что если начальное приближение находится далеко от корня, то число итераций при этом увеличивается, а объем вычислений возрастает.

Метод Ньютона использует значения функции и ее первой производной, что повышает скорость сходимости до квадратичной, но он сходится не при любых начальных приближениях, что накладывает дополнительные ограничения на функцию, ее первую и вторую производные.

2 Интерполяция функций

Пусть функция $f(x)$ задана таблицей значений x_i, y_i на интервале $[a;b]$:

$$y_i = f(x_i), \quad i = \overline{0,n}, \quad a \le x_i \le b. \tag{2.1}$$

Задача интерполяции – найти функцию $F(x)$, принимающую в точках x_i те же значения y_i.

Условие интерполяции:

$$F(x_i) = y_i \ . \tag{2.2}$$

При этом предполагается, что среди значений x_i нет одинаковых. Точки x_i называют узлами интерполяции.

Если $F(x)$ ищется только на отрезке $[a;b]$, то это задача интерполяции, а если за пределами первоначального отрезка, то это задача экстраполяции. Таким образом, можно ввести следующие определения:

интерполяция (в узком смысле) – вычисление промежуточных значений функции по известному дискретному набору значений функции на заданном интервале;

экстраполяция – вычисление значений функции за пределами первоначально известного интервала;

аппроксимация – определение в явном виде параметров функции, описывающей распределение точек.

Задача нахождения интерполяционной функции $F(x)$ имеет много решений, так как через заданные точки x_i, y_i можно провести бесконечно много кривых, каждая из которых будет графиком функции, для которой выполнены все условия интерполяции. Для практики важен случай аппроксимации функции многочленами:

$$F(x) = P_m(x_i) = a_0 + a_1 \cdot x + a_2 \cdot x^2 + \ldots + a_m \cdot x^m, \quad i = 0,1,\ldots,m. \tag{2.3}$$

При этом искомый полином называется интерполяционным полиномом.

При построении одного многочлена для всего рассматриваемого интервала $[a;b]$, для нахождения коэффициентов многочлена необходимо использовать все уравнения системы (2.3). Данная система содержит n+1 уравнение, следовательно, с ее помощью можно определить n+1 коэффициент. Поэтому максимальная степень интерполяционного многочлена $m = n$, и многочлен принимает вид:

$$P_n(x_i) = a_0 + a_1 \cdot x + a_2 \cdot x^2 + \ldots + a_n \cdot x^n, \quad i = 0,1,\ldots,n \tag{2.4}$$

2.1 Локальная и глобальная интерполяция

Если задан n+1 узел интерполяции, то на этих узлах можно построить один интерполяционный многочлен n-ой степени, n-1 многочленов первой степени и большой набор многочленов степени меньше n, опирающиеся на некоторые из этих узлов.

Теоретически максимальную точность обеспечивает многочлен более высокой степени. Однако на практике наиболее часто используют многочлены невысоких степеней, во избежание погрешностей при расчетах коэффициентов при больших степенях многочлена, а также из-за возможных колебаний функции между узлами.

Если функция $f(x)$ интерполируется на отрезке $[a;b]$ с помощью единого многочлена $P_m(x)$ для всего отрезка, то такую интерполяцию называют глобальной. В случае локальной интерполяции на каждом интервале $[x_i; x_{i+1}]$ строится отдельный интерполяционный полином невысокой степени.

2.2 Кусочно-линейная интерполяция

Простейшим и часто используемым видом локальной интерполяции является линейная (или кусочно-линейная) интерполяция. Она заключается в том, что узловые точки соединяются отрезками прямых (рисунок 2.1), то есть через каждые две точки (x_i, y_i) и (x_{i+1}, y_{i+1}) проводится полином первой степени:

$$F(x) = a_0 + a_1 \cdot x, \quad x_i \le x \le x_{i+1} \tag{2.5}$$

Коэффициенты a_0 и a_1 разные на каждом интервале $[x_i; x_{i+1}]$ и находятся из выполнения условий интерполяции на концах отрезка:

$$\begin{cases} y_i = a_0 + a_1 \cdot x_i ; \\ y_{i+1} = a_0 + a_1 \cdot x_{i+1}. \end{cases} \tag{2.6}$$

Из системы уравнений (2.6) можно найти коэффициенты a_0 и a_1 и получить уравнение прямой на каждом интервале.

Линейная интерполяция является частным случаем многочлена Лагранжа, построенного по двум точкам, поэтому вместо прямого вычисления коэффициентов можно воспользоваться формулой:

$$F(x) = y_i \frac{x - x_{i+1}}{x_i - x_{i+1}} + y_{i+1} \frac{x - x_i}{x_{i+1} - x_i} \ . \tag{2.7}$$

При использовании кусочно-линейной интерполяции сначала нужно определить интервал i, в который попадает расчетное значение x, а затем в выражение (2.7) подставить значения (x_i, y_i) и (x_{i+1}, y_{i+1}) для данного интервала.

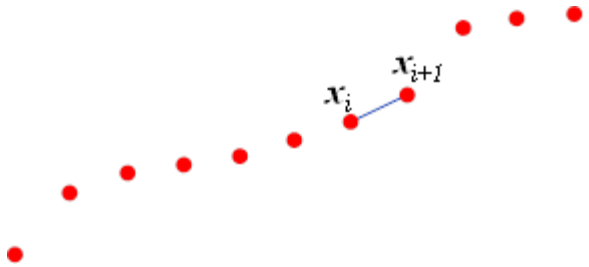


Рисунок 2.1 – Кусочно-линейная интерполяция

2.3 Кусочно-квадратичная интерполяция

В случае квадратичной интерполяции, для каждых трех узловых точек (x_i, y_i), (x_{i+1}, y_{i+1}), (x_{i+2}, y_{i+2}) строится уравнение параболы (рисунок 2.2):

$$F(x) = a_0 + a_1 \cdot x + a_2 \cdot x^2, \quad x_i \le x \le x_{i+2} \tag{2.8}$$

Здесь коэффициенты a_0, a_1 и a_2 разные на каждом интервале $[x_i, x_{i+2}]$ и определяются решением системы уравнений для условия прохождения параболы через три точки:

$$\begin{cases} y_i = a_0 + a_1 \cdot x_i + a_2 \cdot x_i^2; \\ y_{i+1} = a_0 + a_1 \cdot x_{i+1} + a_2 \cdot x_{i+1}^2; \\ y_{i+2} = a_0 + a_1 \cdot x_{i+2} + a_2 \cdot x_{i+2}^2. \end{cases} \tag{2.9}$$

Квадратичная интерполяция также является частным случаем многочлена Лагранжа, построенного по трем точкам, поэтому вместо прямого вычисления коэффициентов можно воспользоваться формулой:

$$F(x) = y_i \frac{(x - x_{i+1})(x - x_{i+2})}{(x_i - x_{i+1})(x_i - x_{i+2})} + y_{i+1} \frac{(x - x_i)(x - x_{i+2})}{(x_{i+1} - x_i)(x_{i+1} - x_{i+2})} + y_{i+2} \frac{(x - x_i)(x - x_{i+1})}{(x_{i+2} - x_i)(x_{i+2} - x_{i+1})} \tag{2.10}$$

При использовании кусочно-линейной интерполяции сначала нужно определить интервал i, в который попадает расчетное значение x, а затем в выражение (2.10) подставить значения $(x_i, y_i), (x_{i+1}, y_{i+1}), (x_{i+2}, y_{i+2})$ для данного интервала.

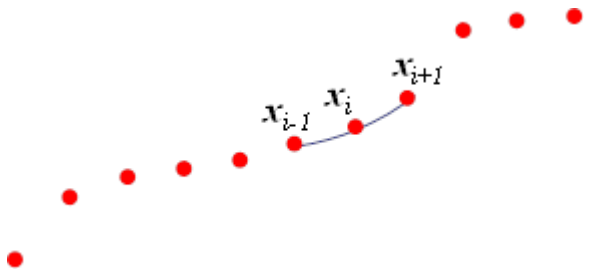


Рисунок 2.2 – Кусочно-квадратичная интерполяция

2.4 Кубические сплайны

Кубическим сплайном называется функция $S(x)$, которая удовлетворяет следующим требованиям:

на каждом i-ом локальном отрезке интерполяции между соседними заданными узлами $[x_i, x_{i+1}], i=\overline{1,n}$ она является многочленом третьей степени;

имеет непрерывные первую и вторую производные на всём интервале интерполяции $[x_1, x_{n+1}]$;

во всех узлах интерполяции x_i выполняются условия равенства значений исходной табличной функции $f(x)$ и функции $S(x)$, а также их первых производных: $S(x_i)=f(x_i)$ и $S'(x_i)=f'(x_i)$;

выполняются граничные условия для вторых производных: $S''(x_1)=S''(x_{n+1})=0$ (в этом случае существует единственный такой сплайн).

Кубические сплайны делятся на два класса.

Локальные. Используются первые производные $f'(x)$, вычисляемые по трем ближайшим к точке *x* узлам интерполяции. Локальные сплайны ненамного лучше кусочно-линейной или кусочно-квадратичной интерполяции и используются редко.

Глобальные. Основаны на вычислении массива вторых производных $f''(x_i)$ по всем узлам x_i, который рассчитывается заранее путем решения единой трехдиагональной СЛАУ. Такие сплайны достаточно хороши и являются одним из основных методов интерполяции. В отличие от кусочно-линейной и кусочно-квадратичной интерполяции они дают гладкие кривые за счет непрерывности первой и второй производных. К недостаткам относится то, что на каждом интервале расчетная формула своя.

Пусть имеется таблица значений $f(x_i)$, где $i=\overline{1,(n+1)}$, то есть у нас есть $(n+1)$ узел интерполяции и n интервалов между ними. В общем случае шаг сетки узлов может быть неравномерным.

Для нахождения значения сплайна в некоторой введенной точке необходимо сначала проверить, принадлежит ли данная точка общему интервалу интерполяции $[x_1, x_{n+1}]$. Если принадлежит, то нужно определить, между какими заданными узлами находится введенная точка, то есть найти номер i подынтервала $x\in[x_i, x_{i+1}], i=\overline{1,n}$.

Формула вычисления значения сплайна в точке имеет вид:

$$F(x)=\frac{1}{6h_i}\left[m_i(x_{i+1}-x)^3+m_{i+1}(x-x_i)^3\right]+\frac{1}{h_i}\left[\left(f_i-\frac{m_i h_i^2}{6}\right)(x_{i+1}-x)+\left(f_{i+1}-\frac{m_{i+1}h_i^2}{6}\right)(x-x_i)\right], \quad (2.11)$$

где $h_i=x_{i+1}-x_i, \quad i=\overline{1,n}$ – расстояния между соседними узлами;

$m_i=f''(x_i), \quad i=\overline{1,(n+1)}$ – значения вторых производных в узловых точках.

Вторые производные m_i находятся из трех диагональной СЛАУ:

$$h_i m_i+2(h_i+h_{i+1})m_{i+1}+h_{i+1}m_{i+2}=6\left(\frac{f_{i+2}-f_{i+1}}{h_{i+1}}-\frac{f_{i+1}-f_i}{h_i}\right), \quad i=\overline{1,(n-1)}. \quad (2.12)$$

Всего в этой системе $(n-1)$ уравнение и $(n+1)$ неизвестных вторых производных $m_1, m_2, \ldots, m_{n+1}$, поэтому система не полностью определяет m_i. Сведем ее к трехдиагональной СЛАУ заданием граничных условий. Для нормального сплайна $m_1 = m_{n+1} = 0$. Теперь в СЛАУ $(n-1)$ неизвестная $m_2, m_3, \ldots, m_n$ и она является трехдиагональной. Такая система эффективнее всего решается методом прогонки.

Если введенная точка не принадлежит общему интервалу интерполяции $[x_1, x_{n+1}]$, то для расчета значений функции за его пределами можно воспользоваться методами экстраполяции.

Простейший способ – использовать линейную экстраполяцию. В этом случае асимптотическое поведение функции вне интервала описывается формулами линейной экстраполяции влево (2.13) или вправо (2.14):

$$F(x) = f_1 + \left[-\frac{x_2 - x_1}{6} m_2 + \frac{f_2 - f_1}{x_1 - x_0} \right] (x - x_1), \quad x < x_1; \tag{2.13}$$

$$F(x) = f_{n+1} + \left[\frac{x_{n+1} - x_n}{6} m_n + \frac{f_{n+1} - f_n}{x_{n+1} - x_n} \right] (x - x_{n+1}), \; x > x_{n+1}. \tag{2.14}$$

2.5 Многочлен Лагранжа

Пусть задана система точек $x_i, i = \overline{1, n+1}$, в которых известны значения функции $y_i = f(x_i)$.

Рассмотрим пример построения интерполяционного многочлена Лагранжа по заданной системе точек (в общем случае для неравноудаленных аргументов). Построим некоторый многочлен $P_n(x)$ таким образом, чтобы его значения совпали со значениями функции, заданными в таблице, для тех же аргументов, то есть $P_n(x_i) = y_i, \quad i = \overline{1, n+1}$. Лагранж предложил строить многочлен степени *n* в виде:

$$\begin{aligned} P_n(x) = a_1 (x - x_2) \ldots (x - x_{n+1}) + a_2 (x - x_1)(x - x_3) \ldots (x - x_{n+1}) + \ldots + \\ + a_{n+1} (x - x_1) \ldots (x - x_n). \end{aligned} \tag{2.15}$$

Здесь в каждом слагаемом коэффициенту a_i соответствует разность $x - x_i$. Найдем неизвестные коэффициенты $a_i, i = \overline{1, n+1}$, называемые коэффициентами Лагранжа, используя условие $P_n(x) = y_i, \quad i = \overline{1, n+1}$.

При $x = x_1$:

$$P_n(x_1) = y_1;$$

$$P_n(x_1) = a_1 (x_1 - x_2)(x_1 - x_3) \ldots (x_1 - x_{n+1}) = y_1.$$

Следовательно, коэффициент a_1 вычисляется по следующей формуле:

$$a_1 = \frac{y_1}{(x_1 - x_2)(x_1 - x_3) \ldots (x_1 - x_{n+1})}.$$

При $x = x_2$:

$$P_n(x_2) = y_2;$$

$$P_n(x_2) = a_2(x_2 - x_1)(x_2 - x_3)\ldots(x_2 - x_{n+1}) = y_2.$$

Следовательно, коэффициент a_1 вычисляется по формуле:

$$a_2 = \frac{y_2}{(x_2 - x_1)(x_2 - x_3)\ldots(x_2 - x_{n+1})}.$$

Значения остальных коэффициентов вычисляются аналогично.

С учетом найденных коэффициентов интерполяционный многочлен Лагранжа запишется в виде:

$$P_n(x) = y_1 \frac{(x - x_2)(x - x_3)\times\ldots\times(x - x_{n+1})}{(x_1 - x_2)\times\ldots\times(x_1 - x_{n+1})} + \ldots + y_{n+1} \frac{(x - x_1)(x - x_2)\times\ldots\times(x - x_n)}{(x_{n+1} - x_1)\times\ldots\times(x_{n+1} - x_n)}. \quad (2.16)$$

Погрешность формулы (2.16) определяется остаточным членом:

$$R_n(x) = \frac{f^{n+1}(\xi)}{(n+1)!}(x - x_1)\ldots(x - x_{n+1}), \quad (2.17)$$

где ξ – точка наименьшего промежутка, содержащего все узлы x_i, $i = \overline{1, n+1}$ и точку x.

Используя условные обозначения суммы и произведения, формулу (2.16) можно записать в сокращенном виде:

$$P_n(x) = \sum_{i=1}^{n+1} \left[y_i \cdot \prod_{\substack{j=1, \\ j \neq i}}^{n+1} \frac{x - x_j}{x_i - x_j} \right]. \quad (2.18)$$

В отличие от кусочно-линейной, кусочно-квадратичной интерполяции и кубического сплайна, формула многочлена Лагранжа будет общая для всего интервала $[x_1, x_{n+1}]$, поэтому определять номер интервала $[x_i, x_{i+1}]$ для каждой расчетной точки не нужно.

2.6 Кусочно-квадратичная интерполяция функции по таблице значений

Задание: с помощью кусочно-квадратичной интерполяции построить график приближенной функции, проходящей через экспериментальные точки, представленные в таблице 2.1, и вычислить значения неизвестной функции в промежуточных точках.

Таблица 2.1 – Экспериментальные данные для интерполяции

11		Экспериментальные точки										Точки вычисления $f(x)$							
1	x	-9,5	-7	-5,5	-3	-1	2,5	3,5	6	8	10	-8	-6	-4	-2	2	3	5	9
	y	-3	-1	2	3,5	3,75	1,5	-0,5	-2,75	-3,25	-3,5								
2	x	-9	-3,5	0	0,5	0,75	1,5	5,5	8,5	9		-10	-5,5	-1	1	3	6	7,5	
	y	-4	-1	3,5	5,5	8	5,5	0,5	-4	-7									

С помощью среды Python е напишем программу для интерполяции.

Проект будет включать в себя скрипт, в котором вводятся исходные данные, выполняется построение графика для отделения корней и выводится результат (листинг 2.1);

Gauss(A, b) – Функция, которая реализует метод Гаусса для решения системы линейных уравнений.

Interval(x, x1) – Функция, которая вычисляет интервал, в который попадает каждая точка из x1 на основе значений из x. Результат представляет собой массив индексов интервалов, в которые попадает каждая точка из x1.

Sguare_val(x, y, x1, itr) – Функция, которая реализует кусочно-квадратичную интерполяцию для заданных экспериментальных точек (x, y) и новых точек x1, используя вычисленные интервалы itr.

```
import numpy as np
import matplotlib.pyplot as plt

def Gauss(A, b):
    n = A.shape[0]

    # Прямой ход метода Гаусса с выбором ведущего элемента по
столбцу
    if np.linalg.det(A) != 0:
        matrix = np.column_stack((A, b))

        k = 0
        while k < n:
            # Выбор ведущего элемента по столбцу
            max_row = k
            for i in range(k+1, n):
                if abs(matrix[i, k]) > abs(matrix[max_row, k]):
                    max_row = i
            if max_row != k:
                matrix[[k, max_row]] = matrix[[max_row, k]]

            j = n
            while j >= k:
                matrix[k, j] = matrix[k, j] / matrix[k, k]
                j -= 1

            i = k + 1
            while i < n:
                j = n
                while j >= k:
                    matrix[i, j] = matrix[i, j] - matrix[i, k] *
matrix[k, j]
                    j -= 1
                i += 1

            k += 1

        # Обратный ход метода Гаусса
        x = np.zeros(n)
        x[n - 1] = matrix[n - 1, n]
```

Листинг 2.1 – Реализация кусочно-квадратичной интерполяции, лист 1

```
        i = n - 2
        while i >= 0:
            j = i + 1
            x[i] = matrix[i, n]

            while j < n:
                x[i] = x[i] - matrix[i, j] * x[j]
                j += 1
            i -= 1

        return x

def interval(x, x1):
    n = len(x)
    n1 = len(x1)
    itr = np.zeros(n1)
    k = 0
    i = 0


    while i < n1:
        if x1[i] < x[0]:
            itr[i] = 0
        j = k

        while j <= n - 2:
            if x[j] <= x1[i] and x1[i] <= x[j+1]:
                itr[i] = j
                k = j
            j = j + 1

        if x1[i] > x[n-1]:
            itr[i] = n
        i = i + 1
    return itr

def square_val(x, y, x1, itr):
    i = 0
    n = len(x)
    n1 = len(x1)
    a = np.zeros(100)
    b = np.zeros(100)
    c = np.zeros(100)
    y1 = np.zeros(n1)

    while i < n - 2:
        X = np.array([[x[i] ** 2, x[i], 1], [x[i + 1] ** 2, x[i +
1], 1], [x[i + 2] ** 2, x[i + 2], 1]])
```

Листинг 2.1, лист 2

```
        Y = np.array([[y[i]], [y[i + 1]], [y[i + 2]]])

        C = Gauss(X, Y)
        #C = np.linalg.solve(X, Y)
        a[i] = C[0]
        b[i] = C[1]
        c[i] = C[2]
        i = i + 1

    a[n - 2] = a[n - 3]
    b[n - 2] = b[n - 3]
    c[n - 2] = c[n - 3]
    i = 0

    while i < n1:
        j = int(itr[i])
        if (j < 0) or (j > n - 2):
            y1[i] = None
        if (j >= 0) and (j <= n - 2):
            y1[i] = a[j] * x1[i] ** 2 + b[j] * x1[i] + c[j]
        i = i + 1
    return y1

#Первая таблица
# x = np.array([-9.5, -7, -5.5, -3, -1, 2.5, 3.5, 6, 8, 10])
# y = np.array([-3, -1, 2, 3.5, 3.75, 1.5, -0.5, -2.75, -3.25, -
3.5])
# x1 = np.array([-8, -6, -4, -2, 2, 3, 5, 9])

#Вторая таблица
x = np.array([-9, -3.5, 0, 0.5, 0.75, 1.5, 5.5, 8.5, 9])
y = np.array([-4, -1, 3.5, 5.5, 8, 5.5, 0.5, -4, -7])
x1 = np.array([-10, -5.5, -1, 1, 3, 6, 7.5])

x1 = np.sort(x1)
itr = interval(x, x1)
y1 = square_val(x, y, x1, itr)

print("Таблица расчетных узлов")
print(x1)
print(y1)

a = min(np.concatenate((x, x1)))
b = max(np.concatenate((x, x1)))

x2 = np.arange(a, b, 0.01)
itr = interval(x, x2)
y2 = square_val(x, y, x2, itr)
```

Листинг 2.1, лист 3

```
plt.figure()
plt.plot(x, y, 'r*', label='Экспериментальные точки')
plt.plot(x1, y1, 'go', label='Точки вычисления')
plt.plot(x2, y2, label='График')
plt.title('Кусочно-квадратичная интерполяция')
plt.legend()
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.show()
```

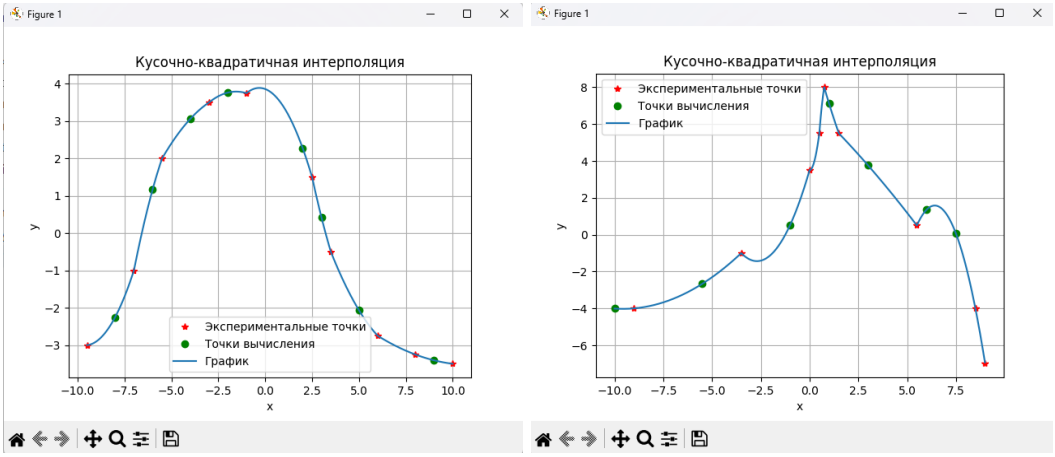
Листинг 2.1, лист 4

Значения функции в расчетных точках приведены в таблице 2.2.

Таблица 2.2 – Значения функции при кусочно-квадратичной интерполяции

		Точки вычисления $f(x)$							
1	x	-8	-6	-4	-2	2	3	5	9
	y	-2.25	1.175	3.05833	3.7646	2.2738	0.4214	-2.0667	-3.4063
2	x	-10	-5.5	-1	1	3	6	7.5	
	y	-4.0108	-2.66667	0.51786	7.1118	3.7589	1.35714	0.07143	

Графики для первой и второй функции, построенные с помощью программы Visual Studio Code, представлены на рисунке 2.3.



а б

Рисунок 2.3 – Квадратичная интерполяция для первой (а) и второй (б) функции

2.7 Интерполяция функции кубическим сплайном по таблице значений

Задание: с помощью интерполяции кубическим сплайном построить график приближенной функции, проходящей через экспериментальные точки, представленные в таблице 2.1, и вычислить значения неизвестной функции в промежуточных точках.

С помощью среды Python напишем программу для интерполяции.

Проект будет включать в себя скрипт, в котором вводятся исходные данные, выполняется построение графика для отделения корней и выводится результат (листинг 2.2);

Mariant(var, table) – Функция, которая возвращает набор точек (x, y) или расчетные узлы x1 в зависимости от значения переменной var.

interval(x, x1) – Функция, которая вычисляет интервал, в который попадает каждая точка из x1 на основе значений из x.

sort_p(x, y) - Функция, которая сортирует массивы точек x и y по значениям x.

progon(x, y) - Функция, которая вычисляет коэффициенты сплайна методом прогонки для заданных точек (x, y).

spline_val(x, y, x1, itr, M) - Функция, которая реализует интерполяцию кубическими сплайнами для заданных точек (x, y) и новых точек интерполяции (x1), используя расчетные узлы и коэффициенты M сплайна.

```
import numpy as np
import matplotlib.pyplot as plt

def variant(var, table):
    if var == 11:
        if table == 1:
            x = [-9.5, -7, -5.5, -3, -1, 2.5, 3.5, 6, 8, 10];
            y = [-3, -1, 2, 3.5, 3.75, 1.5, -0.5, -2.75, -3.25, -
3.5];
            x1 = [-8, -6, -4, -2, 2, 3, 5, 9];
        elif table == 2:
            x = [-9, -3.5, 0, 0.5, 0.75, 1.5, 5.5, 8.5, 6];
            y = [-4, -1, 3.5, 5.5, 8, 5.5, 0.5, -4, -7];
            x1 = [-10, -5.5, -1, 1, 3, 6, 7.5];
    return x, y, x1;

def interval(x, x1):
    n = len(x) - 1;
    n1 = len(x1) - 1;
    itr = [0] * (n1 + 1)
    k = 0;
    i = 0;


    while i <= n1:
        if x1[i] <= x[0]:
            itr[i] = -1;
            i = i + 1;
            continue
        j = k;

        while j <= n - 1:
            if x[j] <= x1[i] and x1[i] <= x[j + 1]:
                itr[i] = j;
                k = j;
                break;
            j = j + 1;
```

Листинг 2.2 – Реализация интерполяции функции кубическим сплайном, лист 1

```
        if x1[i] > x[n]:
            itr[i] = n + 1;
        i = i + 1;
    return itr;

def sort_p(x, y):
    n = len(x);
    for i in range(n - 1):
        if x[i] > x[i + 1]:
            temp = x[i];
            x[i] = x[i + 1];
            x[i+1] = temp;
            temp = y[i];
            y[i] = y[i + 1];
            y[i + 1] = temp;
    return x, y;
def progon(x, y):
    n = len(x) - 1;
    h = [(x[i + 1] - x[i]) for i in range(n)]
    A = [0] * (n - 1);
    B = [0] * (n - 1);
    C = [0] * (n - 1);

    for i in range(n - 1):
        A[i] = h[i];
        B[i] = 2 * (h[i] + h[i + 1]);
        C[i] = h[i + 1];
    A[0] = 0;
    C[n - 2] = 0;

    D = [6 * ((y[i + 2] - y[i + 1]) / h[i + 1] - (y[i + 1] - y[i])
/ h[i]) for i in range(n - 1)];
    Q = [0] * n;
    R = [0] * n;

    for i in range(n - 1):
        Q[i + 1] = -(C[i] / (B[i] + A[i] * Q[i]));
        R[i + 1] = (D[i] - A[i] * R[i]) / (B[i] + A[i] * Q[i]);

    M = [0] * (n - 1);
    M[n - 2] = R[n - 1];
    for i in range(n - 3, -1, -1):
        M[i] = Q[i + 1] * M[i + 1] + R[i + 1];
    M = [0] + M + [0];
    return M;

def spline_val(x, y, x1, itr, M):
    n = len(x) - 1
```

Листинг 2.2, лист 2

```
    n1 = len(x1)
    y1 = [0] * (n1)
    h = [x[i + 1] - x[i] for i in range(n)]
    i = 0
    while i < n1:
        j = itr[i]
        if j == -1:
            y1[i] = y[0] + ((x[0] - x[1]) * M[1] / 6 + (y[1] -
y[0]) / (x[1] - x[0])) * (x1[i] - x[0])
            i = i + 1
        elif j >= 0 and j <= n:
            y1[i] = (1 / (6 * h[j]) * ((M[j] * (x[j + 1] -x1[i])
** 3) + M[j + 1] * (x1[i] - x[j]) ** 3)+ 1 / h[j] * ((y[j] -
((M[j] * h[j] ** 2) /6)) * (x[j + 1] - x1[i]) + (y[j + 1] - ((M[j
+ 1]* h[j] ** 2) / 6)) * (x1[i] - x[j])))
            i = i + 1
        elif j == n + 1:
            y1[i] = y[n] + ((x[n] - x[n - 1]) * M[n - 1] / 6 +
(y[n] - y[n - 1]) / (x[n] - x[n - 1])) * (x1[i] - x[n])
            i = i + 1
    return y1
var = 11;
for i in range (1, 3):
    x, y, x1 = variant(var, i);
    x, y = sort_p(x, y);
    x1 = np.sort(x1);
    M = progon(x, y);
    itr = interval(x, x1);
    y1 = spline_val(x, y, x1, itr, M);
    print(f'Таблица {i}');
    print(x1);
    print(y1);
    a = min(np.concatenate((x, x1)))
    b = max(np.concatenate((x, x1)))
    step = 0.02
    x2 = np.arange(a, b, step)
    itr = interval(x, x2);
    y2 = spline_val(x, y, x2, itr, M);
    plt.figure(1);
    plt.plot(x2, y2, 'c', label='Кривая интерполяции');
    plt.plot(x, y, 'bo', label='Заданные точки');
    plt.plot(x1, y1, 'r*', label='Расчетные узлы');
    plt.title('Кубический сплайн');
    plt.legend();
    plt.xlabel('x');
    plt.ylabel('y');
    plt.grid(True);
    plt.show();
```

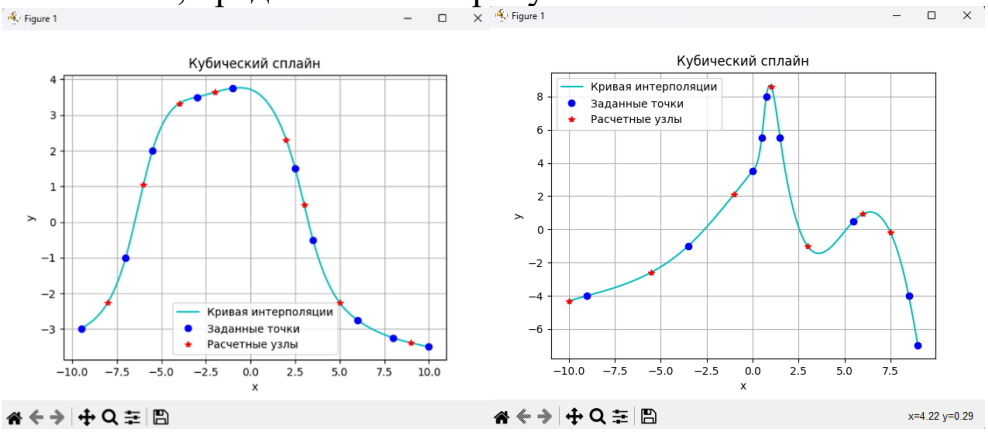
Листинг 2.2, лист 3

Значения функции в расчетных точках приведены в таблице 2.3.

Таблица 2.3 – Значения функции при интерполяции кубическим сплайном

		Точки вычисления $f(x)$							
1	x	-8	-6	-4	-2	2	3	5	9
	y	-2.256	1.052	3.323	3.639	2.308	0.496	-2.265	-3.390
2	x	-10	-5.5	-1	1	3	6	7.5	
	y	-4.309	-2.583	2.117	8.615	-1.016	-0.932	-0.176	

Графики для первой и второй функции, построенные с помощью программы VS Code, представлены на рисунке 2.4.



а б

Рисунок 2.4 – Интерполяция сплайном для первой (а) и второй (б) функции

2.8 Интерполяция функции многочленом Лагранжа по таблице значений

Задание: с помощью интерполяции многочленом Лагранжа построить график приближенной функции, проходящей через экспериментальные точки, представленные в таблице 2.1, и вычислить значения неизвестной функции в промежуточных точках.

С помощью среды Python напишем программу для интерполяции.

Проект будет включать в себя скрипт, в котором вычисляется значение функции в промежуточных точках с помощью интерполяции многочленом Лагранжа и строится график (листинг 2.3);

lagrange_val(x, y, x_calc): Эта функция рассчитывает значения интерполяционного полинома Лагранжа в заданных точках x_calc для заданных узлов x и значений функции в узлах y. Она использует формулу интерполяционного полинома Лагранжа для расчета значений в указанных точках.

lagrange_main(x, y, x_calc): Это главная функция, которая сначала сортирует x_calc, затем вызывает lagrange_val для получения значений интерполяционного полинома Лагранжа в точках x_calc и выводит их на экран. Затем она вычисляет значения интерполяционного полинома Лагранжа для построения графика, используя более плотное множество точек, чем исходные узлы x. На последнем этапе она строит график, отображающий исходные узлы x и y, а также вычисленные значения интерполяционного полинома Лагранжа для заданных точек x_calc.

```
import numpy as np
from matplotlib import pyplot as plt
def lagrandge_val(x, y, x_calc):
    dlina_x = len(x) - 1
    dlina_x_calc = len(x_calc) - 1
    y_calc = [0] * (dlina_x_calc + 1)
    k = 0
    while k <= dlina_x_calc:
        # выход за границы
        if x_calc[k] < x[0] or x_calc[k] > x[dlina_x]:
            y_calc[k] = None
            k = k + 1
            continue
        i = 0
        while i <= dlina_x:
            P = 1
            j = 0
            while j <= dlina_x:
                if i != j:
                    P = P * (x_calc[k] - x[j]) / (x[i] - x[j])
                j += 1
            y_calc[k] += P * y[i]
            i = i + 1
        k = k + 1
    return y_calc

def lagrandge_main(x, y, x_calc):
    x_calc.sort()
    y_calc = lagrandge_val(x, y, x_calc)
    print(str(x_calc) + "\n" + str(y_calc))
    # x и y для линии
    dlina_x = len(x) - 1
    buff_x = [x for x in np.arange(x[0], x[dlina_x] + 0.01, 0.01)]
    buff_y = lagrandge_val(x, y, buff_x)
    # график
    plt.plot(buff_x, buff_y, '-', x, y, 'o', x_calc, y_calc, 'o')
    plt.grid(True)
    plt.show()
x1 = np.array([-9.5, -7, -5.5, -3, -1, 2.5, 3.5, 6, 8, 10])
y1 = np.array([-3, -1, 2, 3.5, 3.75, 1.5, -0.5, -2.75, -3.25, -
3.5])
x11 = np.array([-8, -6, -4, -2, 2, 3, 5, 9])
x2 = np.array([-9, -3.5, 0, 0.5, 0.75, 1.5, 5.5, 8.5, 9])
y2 = np.array([-4, -1, 3.5, 5.5, 8, 5.5, 0.5, -4, -7])
x22 = np.array([-10, -5.5, -1, 1, 3, 6, 7.5])
lagrandge_main(x1, y1, x11)
lagrandge_main(x2, y2, x22)
```

Листинг 2.4 – Реализация интерполяции многочленом Лагранжа

Значения функции в расчетных точках приведены в таблице 2.4.

Задание: с помощью интерполяции многочленом Лагранжа построить график приближенной функции, проходящей через экспериментальные точки, представленные в таблице 2.1, с помощью Метода Гаусса, и вычислить значения неизвестной функции в промежуточных точках.

С помощью среды Python напишем программу для интерполяции.

Проект будет включать в себя скрипт, в котором вычисляется значение функции в промежуточных точках с помощью интерполяции многочленом Лагранжа, с помощью Метода Гаусса, и строится график (листинг 2.3);

Функция variant(var, table): Эта функция принимает переменную var и table, и возвращает набор значений x, y, и x1 в соответствии со значениями var и table. Значения x представляют точки x, y - значения функции в узлах x, и x1 - аргументы для интерполяции.

Функция Gauss(A, b): Это реализация метода Гаусса для решения систем линейных уравнений. Она принимает матрицу коэффициентов A и вектор свободных членов b, использует метод Гаусса с выбором ведущего элемента по столбцу для решения системы, и возвращает решение x.

```
import numpy as np
import matplotlib.pyplot as plt
np.set_printoptions(suppress=True)

import numpy as np
import matplotlib.pyplot as plt
np.set_printoptions(suppress=True)
def variant(var, table):
    if var == 11:
        if table == 1:
            x = np.array([-9.5, -7, -5.5, -3, -1, 2.5, 3.5, 6, 8,
10])
            y = np.array([-3, -1, 2, 3.5, 3.75, 1.5, -0.5, -2.75,
-3.25, -3.5])
            x1 = np.array([-8, -6, -4, -2, 2, 3, 5, 9])
        elif table == 2:
            x = np.array([-9, -3.5, 0, 0.5, 0.75, 1.5, 5.5, 8.5,
9])
            y = np.array([-4, -1, 3.5, 5.5, 8, 5.5, 0.5, -4, -7])
            x1 = np.array([-10, -5.5, -1, 1, 3, 6, 7.5])
    return x, y, x1;
def Gauss(A, b):
    # прямой ход метода Гаусса с выбором ведущего элемента по
столбцу
    if np.linalg.det(A) != 0:
        matrix = np.column_stack((A, b))
        # print(matrix)

        k = 0
        n = A.shape[0]
```

Листинг 2.5 – Реализация интерполяции многочленом Лагранжа методом Гаусса, лист 1

```
        while k < n:
            # выбор ведущего элемента по столбцу
            max_row = k
            for i in range(k + 1, n):
                if abs(matrix[i, k]) > abs(matrix[max_row, k]):
                    max_row = i
            if max_row != k:
                matrix[[k, max_row]] = matrix[[max_row, k]]

            j = n
            while j >= k:
                matrix[k, j] = matrix[k, j] / matrix[k, k]
                j -= 1

            i = k + 1
            while i < n:
                j = n
                while j >= k:
                    matrix[i, j] = matrix[i, j] - matrix[i, k] *
matrix[k, j]
                    j -= 1
                i += 1

            k += 1

        # обратный ход метода Гаусса
        x = np.zeros(n)
        x[n - 1] = matrix[n - 1, n]
        i = n - 2
        while i >= 0:
            j = i + 1
            x[i] = matrix[i, n]
            while j < n:
                x[i] = x[i] - matrix[i, j] * x[j]
                j += 1
            i -= 1
        return x

var = 11;
for k in range (1,3):
    x, y, x1 = variant(var, k);
    n = len(x)
    j = 0
    xK = np.zeros((n, n))
    while j < n:
        i = 0
        while i < n:
            xK[i, j] = x[i]**(n-j - 1)
```

Листинг 2.5, лист 2

```
            i = i + 1
        j = j + 1
    # print(xK)

    y = y.reshape(-1, 1) # делает двумерный массив, чтобы значения
были вертикальными
    # print(y)
    a = Gauss(xK, y)
    # print(a)
    i = 0
    n1 = len(x1)
    y1 = np.zeros(n1)
    while i < n1:
        j = 0
        while j < n:
            y1[i] = y1[i] + a[j] * x1[i]**(n-j-1)
            j = j + 1
            if x1[i] < x[0] or x1[i] > x[n - 1]:
                y1[i] = None;
        i = i + 1
    print(f'Таблица {k}');
    print(x1)
    print(y1)
    c = min(x)
    b = max(x)
    x2 = np.arange(c, b + 0.01, 0.01)
    i = 0
    n2 = len(x2)
    y2 = np.zeros(n2)
    while i < n2:
        j = 0
        while j < n:
            y2[i] = y2[i] + a[j] * x2[i]**(n-j-1)
            j = j + 1
        i = i + 1

    plt.figure(1);
    plt.plot(x2, y2, 'c', label='Кривая интерполяции');
    plt.plot(x, y, 'bo', label='Заданные точки');
    plt.plot(x1, y1, 'r*', label='Расчетные узлы');
    plt.title('Многочлен Лагранжа')
    plt.legend()
    # plt.xlim(-10, 10)
    # plt.ylim(-10, 10)
    plt.xlabel('x')
    plt.ylabel('y')
    plt.grid(True)
    plt.show()
```

Листинг 2.5, лист 3

Значения функции в расчетных точках приведены в таблице 2.5.

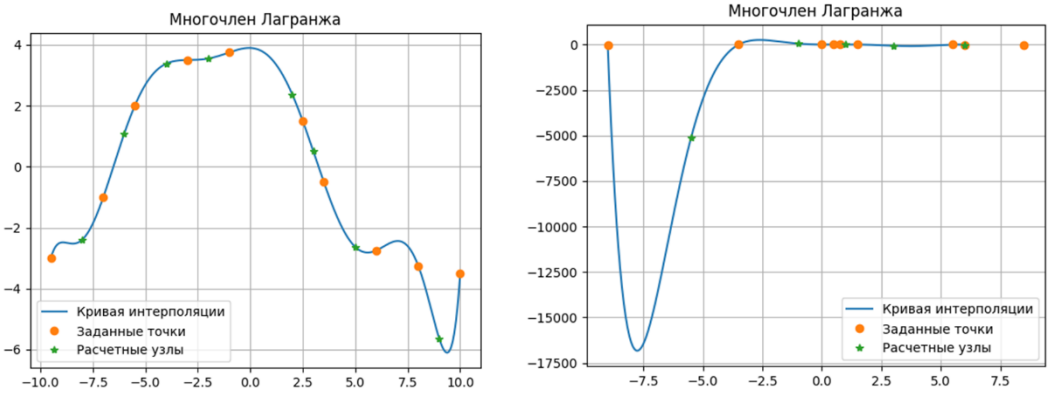
Таблица 2.4 – Значения функции при интерполяции многочленом Лагранжа

		Точки вычисления $f(x)$							
1	x	–8	-6	-4	-2	2	3	5	9
	y	-2.392	1.093	3.382	3.549	2.374	0.511	-2.633	-5.6398
2	x	-10	-5.5	-1	1	3	6	7.5	
	y	None	-4016.4	49.014	9.589	-82.6695	92.851	198.358	

Таблица 2.5 – Значения функции при интерполяции многочленом Лагранжа методом Гаусса

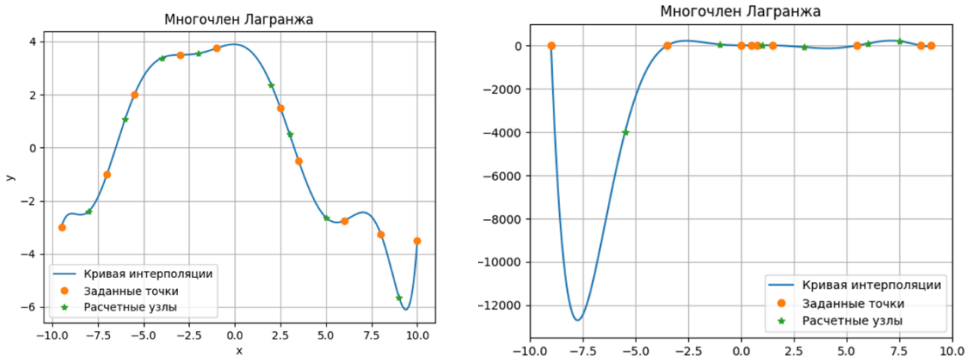
		Точки вычисления $f(x)$							
1	x	–8	-6	-4	-2	2	3	5	9
	y	-2.392	1.093	3.382	3.549	2.374	0.511	-2.633	-5.6398
2	x	-10	-5.5	-1	1	3	6	7.5	
	y	48103.0	-4016.4	49.014	9.589	-82.6695	92.851	198.358	

Графики для первой и второй функции, построенные с помощью программы Visual Studio Code, представлены на рисунках 2.5-2.6.



а б

Рисунок 2.5 – Многочлен Лагранжа для первой (а) и второй (б) функции



а б

Рисунок 2.6 – Многочлен Лагранжа методом Гаусса для первой (а) и второй (б) функции

2.9 Вывод

Среди рассмотренных видов интерполяции можно сделать вывод, что кубический сплайн – это самый точный метод. Использование многочленов третьей степени и согласование значений производных в узлах позволяет получить гладкую кривую без значительных отклонений.

Квадратичная интерполяция основана на уравнении параболы. Она не учитывает значения производных, поэтому дает более высокую погрешность. Минусом данного метода является то, что график функции получается ломаным.

Интерполяция многочленом Лагранжа является худшим несмотря на то, что является более простым в понимании и реализации, так как используется одна форума на всем интервале интерполяции. При данном виде интерполяции особенно видна погрешность на краях графика.

3 Решение обыкновенных дифференциальных уравнений

Дифференциальные уравнения являются основным математическим инструментом моделирования и анализа разнообразных явлений и процессов в науке и технике.

Методы их решения подразделяются на два класса:

аналитические методы, в которых решение получается в виде аналитических функций;

численные (приближенные) методы, где искомые интегральные кривые получают в виде таблиц их численных значений.

Применение аналитических методов позволяет исследовать полученные решения методами математического анализа и сделать соответствующие выводы о свойствах моделируемого явления или процесса. К сожалению, с помощью таких методов можно решать достаточно ограниченный круг реальных задач. Численные методы позволяют получить с определенной точностью приближенное решение практически любой задачи.

Решить дифференциальное уравнение

$$\frac{dy}{dx} = f\left(x, y\right). \tag{3.1}$$

численным методом означает, что для заданной последовательности аргументов x_0 , x_1 , x_2 , . . . , x_n и числа $y_0 = y(x_0)$, не определяя аналитического вида функции $y = F\left(x\right)$, найти значения y_1 , y_2 , . . . , y_n, удовлетворяющие начальным условиям:

$$F\left(x_0\right) = y_0 \ , \quad y_k = F\left(x_k\right) \ , \ k = \left(\overline{1 \, , n} \right).$$

3.1 Метод Эйлера

Этот метод является сравнительно грубым и применяется в основном для ориентировочных расчетов. Однако идеи, положенные в основу метода Эйлера, являются исходными для ряда других численных методов.

Пусть дано обыкновенное дифференциальное уравнение (ОДУ) с начальными условиями (задача Коши):

$$y' = f\left(x, y\right), \ y\left(x_0\right) = y_0 . \tag{3.2}$$

и удовлетворяются условия существования и единственности решения.

Требуется найти решение $y(x)$ задачи Коши (3.2) на отрезке $\left[a, b\right]$. Найдем решение в виде таблицы $y(x_i) = y_i, i = \overline{1,n}$. Для этого разобьем отрезок $\left[a, b\right]$ на n равных частей и построим последовательность

$$x_0, x_1 \ , \ . \ . \ . \ , x_n \ , x_i = x_0 + i \cdot h, \quad i = \overline{0, n},$$

где $h = (b - a)/n$ – шаг интегрирования.

Проинтегрируем исходное уравнение на отрезке $\left[x_k, x_{k+1}\right]$:

$$\int_{x_k}^{x_{k+1}} f(x,y)dx = \int_{x_k}^{x_{k+1}} \frac{dy}{dx}dx = y(x)\Big|_{x_k}^{x_{k+1}} = y(x_{k+1}) - y(x_k) = y_{k+1} - y_k$$

Полученное соотношение можно переписать как

$$y_{k+1} = y_k + \int_{x_k}^{x_{k+1}} f(x,y)dx. \qquad (3.3)$$

Если считать подынтегральную функцию постоянной на участке $[x_k, x_{k+1}]$ и равной значению в начальной точке этого интервала $x = x_k$, то получим

$$\int_{x_k}^{x_{k+1}} f(x_k, y_k)dx = f(x_k, y_k) \cdot x \Big|_{x_k}^{x_{k+1}} = f(x_k, y_k)(x_{k+1} - x_k) = hf(x_k, y_k).$$

Подставляя полученный результат в формулу (3.3), получим основную расчетную формулу метода Эйлера:

$$y_{k+1} = y_k + h \cdot f(x_k, y_k), \quad k = \overline{0, n-1}. \qquad (3.4)$$

Так как для вычисления каждого последующего значения y_{k+1} требуется только одно значение y_k, полученное на предыдущем шаге, то метод Эйлера относят к одношаговым методам.

Вычисление значений $y_1, y_2, \ldots, y_n$ осуществляется с использованием формулы (3.4) следующим образом. По заданным начальным условиям $a = x_0$ и y_0, полагая $k = 0$ в выражении (3.4), вычисляется значение

$$y_1 = y_0 + h\, f(x_0, y_0). \qquad (3.5)$$

Далее, определяя значение аргумента x по формуле $x_1 = x_0 + h$, используя найденное значение y_1 и полагая в формуле (3.4) $k = 1$, вычисляем следующее приближенное значение интегральной кривой $y = F(x)$ как

$$y_2 = y_1 + h\, f(x_1, y_1). \qquad (3.6)$$

Поступая аналогичным образом при $k = \overline{2, n-1}$, определяем все остальные значения y_k, в том числе последнее значение

$$y_{k+1} = y_k + h\, f(x_k, y_k). \qquad (3.7)$$

которое соответствует значению аргумента $x_n = b$.

Погрешность метода Эйлера составляет $O(h^2)$.

Вычислив приближенные значения неизвестной функции в точках $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$, для более гладкого представления искомой интегральной кривой $y = F(x)$ можно воспользоваться одним из методов интерполяции, например, кубическим сплайном.

3.2 Метод Эйлера-Коши

Метод Эйлера-Коши, как и классический метод Эйлера, относится к одношаговым методам, но каждый шаг включает два этапа.

На первом этапе выполняется приблизительная оценка значения функции по обычной формуле Эйлера:

$$y_{k+1}^{*} = y_k + h \cdot f\left(y_k, x_k\right). \tag{3.8}$$

На втором этапе полученное значение используется для более точного расчета по формуле:

$$y_{k+1} = y_k + \frac{h}{2} \cdot \left[f\left(y_k, x_k\right) + f\left(y_{k+1}^{*}, x_k + h\right) \right]. \tag{3.9}$$

Погрешность метода Эйлера составляет $O(h^3)$, что на порядок выше, чем у метода Эйлера.

3.3 Решение ОДУ методом Эйлера

Задание: найдите методом Эйлера численное решение ОДУ первого порядка на отрезке от $a = 0$ до $b = 2$ с шагом интегрирования $h = 0{,}2$:

$$\begin{cases} y' = y + 6\cos 2x - 3\sin 2x + 2 \\ y(0) = -1 \end{cases}$$

Получите таблицу приближенных значений неизвестной функции $y(x)$ на отрезке интегрирования $x \in [a; b]$ с постоянным шагом h.

Аппроксимируйте полученные табличные значения кубическим сплайном и постройте приближенный график функции $y(x)$ на отрезке интегрирования, используя скрипты из главы 2. При построении графика шаг интерполяции примите равным $0{,}1 \cdot h$.

С помощью среды Python напишем программу для интерполяции.

Проект будет включать в себя скрипт, в котором аппроксимируются полученные табличные значения кубическим сплайном и строится приближенный график функции y(x) на отрезке интегрирования (листинг 3.1);

fun(y, x) – Функция, представляющая дифференциальное уравнение. Она вычисляет значение дифференциального уравнения для заданных значений y и x.

progon(x, y) – Функция, реализующая метод прогонки для вычисления второй производной (второй разности) для интерполяционного сплайна.

interval(x, x1) – Функция для вычисления интервалов, в которые попадают новые точки интерполяции, относительно известных точек.

spline_val(x, y, x1, itr, M) – Функция для интерполяции полученных данных кубическим сплайном с использованием рассчитанных интервалов и вторых производных сплайна.

```
import numpy as np
import matplotlib.pyplot as plt
```

Листинг 3.1 – Реализация численного решения ОДУ первого порядка методом Эйлера, лист 1

```
def progon(x, y):
    n = len(x) - 1
    h = np.zeros(n)

    A = np.zeros(n - 1)
    B = np.zeros(n - 1)
    C = np.zeros(n - 1)
    h[0:n] = x[1:n + 1] - x[0:n]
    A[0] = 0
    A[1:n - 1] = h[1:n - 1]
    B[0:n - 1] = 2 * (h[0:n - 1] + h[1:n])
    C[0:n - 2] = h[1:n - 1]
    C[n - 2] = 0
    D = np.zeros(n - 1)
    for i in range(0, n - 1):
        D[i] = 6 * ((y[i + 2] - y[i + 1]) / h[i + 1] - (y[i + 1] -
y[i]) / h[i])

    Q = np.zeros(n)
    R = np.zeros(n)
    for i in range(0, n - 1):
        Q[i + 1] = -(C[i] / (B[i] + A[i] * Q[i]))
        R[i + 1] = (D[i] - A[i] * R[i]) / (B[i] + A[i] * Q[i])
    M = np.zeros(n - 1)
    M[n - 2] = R[n - 1]
    for i in range(n - 3, -1, -1):
        M[i] = Q[i + 1] * M[i + 1] + R[i + 1]
    M = np.insert(M, 0, 0)
    M = np.insert(M, n, 0)
    return M

def interval(x, x1):
    n = len(x)
    n1 = len(x1)
    itr = np.zeros(n1)
    k = 0
    i = 0
    while i < n1:
        if x1[i] <= x[0]:
            itr[i] = 0
            i = i + 1
            continue
        j = k
        while j < n - 1:
            if x[j] <= x1[i] and x1[i] <= x[j + 1]:
                itr[i] = j
                k = j
                break
```

Листинг 3.1, лист 2

```
            j = j + 1
        if x1[i] > x[n - 1]:
            itr[i] = n
        i = i + 1
    return itr

def spline_val(x, y, x1, itr, M):
    n = len(x) - 1
    n1 = len(x1)
    y1 = np.zeros(n1)
    h = np.zeros(n)
    h[0:n] = x[1:n + 1] - x[0:n]
    i = 0
    while i < n1:
        j = int(itr[i])
        if j < 0:
            y1[i] = y[0] + (x[0] - x[1]) * M[1] / 6 + (y[1] -
y[0]) / (x[1] - x[0]) * (x1[i] - x[0])
        if j >= 0 and j < n:
            y1[i] = 1 / (6 * h[j]) * ((M[j] * (x[j + 1] - x1[i])
** 3) + M[j + 1] * (x1[i] - x[j]) ** 3) + 1 / h[j] * (
                        (y[j] - ((M[j] * h[j] ** 2) / 6)) * (x[j +
1] - x1[i]) + (
                            y[j + 1] - ((M[j + 1] * h[j] ** 2) /
6)) * (x1[i] - (x[j])))
        if (j >= n):
            y1[i] = y[n] + ((x[n] - x[n - 1]) * M[n] / 6 + (y[n] -
y[n - 1]) / (x[n] - x[n - 1])) * (x1[i] - x[n])
        i = i + 1
    return y1

def fun(y, x):
    f = y + (6 * np.cos(2 * x)) - (3 * np.sin(2 * x)) + 2
    return f
h = 0.2
xstart = 0
xend = 2
x = np.arange(xstart, xend + h, h)
y = np.zeros(len(x))
y[0] = -1
i = 0
for xstep in np.arange(xstart, xend, h):
    y[i + 1] = y[i] + h * fun(y[i], xstep)
    i = i + 1
M = progon(x, y)
a = np.min(x)
b = np.max(x)
x2 = np.arange(a, b + 0.1 * h, 0.1 * h)
itr = interval(x, x2)
```

Листинг 3.1, лист 3

```
y2 = spline_val(x, y, x2, itr, M)
print("Таблица полученных узлов")
print(np.around(x, 2))
print(np.around(y, 2))
plt.plot(x, y, 'ro', x2, y2, 'b')
plt.legend(['Вычисленные точки', 'Интерполяционная функция'])
plt.grid(True)
plt.show()
```

Листинг 3.1, лист 4

Численное решение ОДУ приведено в таблице 3.1.

Таблица 3.1 – Численное решение ОДУ методом Эйлера

k	1	2	3	4	5	6	7	8	9	10	11
x_k	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6	1.8	2
y_k	-1	-0.4	1.75	2.91	3.76	4.28	4.49	4.5	4.47	4.6	5.11

График приближенного решения ОДУ, построенный с помощью интерполяции кубическим сплайном, представлен на рисунке 3.1.

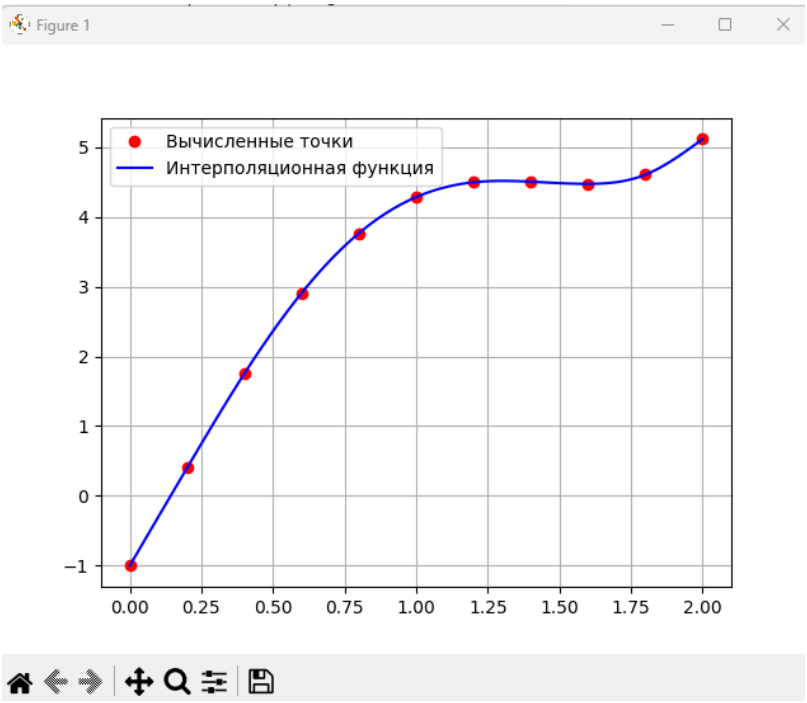


Рисунок 3.1 – График приближенного решения ОДУ методом Эйлера

3.4 Решение ОДУ методом Эйлера-Коши

Задание: найдите методом Эйлера-Коши численное решение ОДУ первого порядка на отрезке от $a = 0$ до $b = 5$ с шагом интегрирования $h = 0{,}5$:

$$\begin{cases} y' = 3x^2 e^{-x} - y \\ y(0) = 0 \end{cases}$$

Получите таблицу приближенных значений неизвестной функции $y(x)$ на отрезке интегрирования $x \in [a; b]$ с постоянным шагом h.

Аппроксимируйте полученные табличные значения кубическим сплайном и постройте приближенный график функции $y(x)$ на отрезке интегрирования, используя скрипты из главы 2. При построении графика шаг интерполяции примите равным $0{,}1 \cdot h$.

С помощью среды Python напишем программу для интерполяции.

Проект будет включать в себя скрипт, в котором аппроксимируются полученные табличные значения кубическим сплайном и строится приближенный график функции y(x) на отрезке интегрирования (листинг 3.2);

progon(x, y) - Эта функция реализует метод прогонки для вычисления коэффициентов кубического сплайна. Она принимает на вход массив точек x и соответствующие значения функции y, затем вычисляет коэффициенты сплайна методом прогонки и возвращает массив коэффициентов сплайна M.

interval(x, x1) - Данная функция определяет интервалы, в которые попадают новые точки интерполяции x1 относительно заданных точек x.

spline_val(x, y, x1, itr, M) - Эта функция выполняет интерполяцию кубическим сплайном для новых точек интерполяции x1 на основе заданных точек x, значений функции y, массива индексов интервалов itr и коэффициентов сплайна M.

fun(y, x) - Функция, представляющая дифференциальное уравнение. Она определяет значение дифференциального уравнения для заданных значений y и x.

```
import numpy as np
import matplotlib.pyplot as plt


def progon(x, y):
    n = len(x) - 1
    h = np.zeros(n)
    A = np.zeros(n - 1)
    B = np.zeros(n - 1)
    C = np.zeros(n - 1)
    h[0:n] = x[1:n + 1] - x[0:n]

    A[0] = 0
    A[1:n - 1] = h[1:n - 1]
    B[0:n - 1] = 2 * (h[0:n - 1] + h[1:n])
    C[0:n - 2] = h[1:n - 1]
    C[n - 2] = 0
    D = np.zeros(n - 1)

    for i in range(0, n - 1):
        D[i] = 6 * ((y[i + 2] - y[i + 1]) / h[i + 1] - (y[i + 1] -
y[i]) / h[i])

    Q = np.zeros(n)
    R = np.zeros(n)
```

Листинг 3.2 – Реализация численного решения ОДУ первого порядка методом Эйлера-Коши, лист 1

```
    for i in range(0, n - 1):
        Q[i + 1] = -(C[i] / (B[i] + A[i] * Q[i]))
        R[i + 1] = (D[i] - A[i] * R[i]) / (B[i] + A[i] * Q[i])
    M = np.zeros(n - 1)
    M[n - 2] = R[n - 1]
    for i in range(n - 3, -1, -1):
        M[i] = Q[i + 1] * M[i + 1] + R[i + 1]
    M = np.insert(M, 0, 0)
    M = np.insert(M, n, 0)
    return M

def interval(x, x1):
    n = len(x)
    n1 = len(x1)
    itr = np.zeros(n1)
    k = 0
    i = 0
    while i < n1:
        if x1[i] <= x[0]:
            itr[i] = 0
            i = i + 1
            continue
        j = k
        while j < n - 1:
            if x[j] <= x1[i] and x1[i] <= x[j + 1]:
                itr[i] = j
                k = j
                break
            j = j + 1
        if x1[i] > x[n - 1]:
            itr[i] = n
        i = i + 1
    return itr

def spline_val(x, y, x1, itr, M):
    n = len(x) - 1
    n1 = len(x1)
    y1 = np.zeros(n1)
    h = np.zeros(n)
    h[0:n] = x[1:n + 1] - x[0:n]
    i = 0
    while i < n1:
        j = int(itr[i])
        if j < 0:
            y1[i] = y[0] + (x[0] - x[1]) * M[1] / 6 + (y[1] -
y[0]) / (x[1] - x[0]) * (x1[i] - x[0])
        if j >= 0 and j < n:
            y1[i] = 1 / (6 * h[j]) * ((M[j] * (x[j + 1] - x1[i])
** 3) + M[j + 1] * (x1[i] - x[j]) ** 3) + 1 / h[j] * (
```

Листинг 3.2, лист 2

```
                        (y[j] - ((M[j] * h[j] ** 2) / 6)) * (x[j +
1] - x1[i]) + (
                            y[j + 1] - ((M[j + 1] * h[j] ** 2) /
6)) * (x1[i] - (x[j])))
        if j >= n:
            y1[i] = y[n] + ((x[n] - x[n - 1]) * M[n] / 6 + (y[n] -
y[n - 1]) / (x[n] - x[n - 1])) * (x1[i] - x[n])
        i = i + 1
    return y1

def fun(y, x):
    f = 3 * (x**2) * np.e**(-x) - y
    return f

h = 0.5
xstart = 0
xend = 5
x = np.arange(xstart, xend + h, h)
y = np.zeros(len(x))
y[0] = -0
i = 0
for xstep in np.arange(xstart, xend, h):
    ytemp = y[i] + h * fun(y[i], xstep)
    y[i + 1] = y[i] + h / 2 * (fun(y[i], xstep) + fun(ytemp, xstep
+ h))
    i = i + 1
M = progon(x, y)
a = np.min(x)
b = np.max(x)
x2 = np.arange(a, b + 0.1 * h, 0.1 * h)
itr = interval(x, x2)
y2 = spline_val(x, y, x2, itr, M)
print("Таблица полученных узлов")
print(np.around(x, 2))
print(np.around(y, 2))
plt.plot(x, y, 'ro', x2, y2, 'b')
plt.legend(['Вычисленные точки', 'Интерполяционная функция'])
plt.grid(True)
plt.show()
```

Листинг 3.2, лист 3

Численное решение ОДУ приведено в таблице 3.2.

Таблица 3.2 – Численное решение ОДУ методом Эйлера-Коши

k	1	2	3	4	5	6	7	8	9	10	11
x_k	0	0.5	1	1.5	2	2.5	3	3.5	4	4.5	5
y_k	0	0.11	0.4	0.77	1.07	1.26	1.32	1.27	1.15	1	0.83

График приближенного решения ОДУ, построенный с помощью интерполяции кубическим сплайном, представлен на рисунке 3.2.

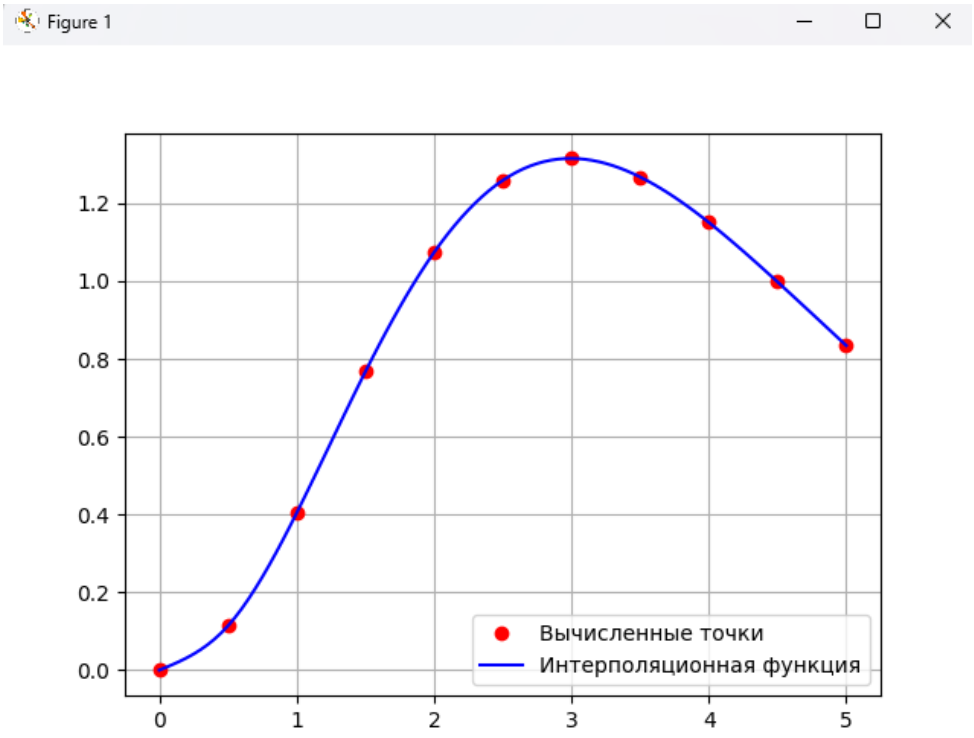


Рисунок 3.2 – График приближенного решения ОДУ методом Эйлера-Коши

3.5 Вывод

Были изучены два метода решения ОДУ первого порядка: метод Эйлера и метод Эйлера-Коши.

В методе Эйлера происходит движение по касательной к интегральной кривой, что может приводить к увеличению погрешности решения на каждом шаге.

Метод Эйлера-Коши, основанный на методе Эйлера, обеспечивает более высокую точность вычислений. Здесь используется среднее значение угла наклона касательных на границах отрезка, что способствует повышению точности решения.

Метод Эйлера использует только значение функции на предыдущем шаге и её производную в этой точке, чтобы вычислить значение функции на следующем шаге. Метод Эйлера-Коши более точен, так как учитывает не только производную в текущей точке, но и изменение производной на следующем шаге.

Заключение

Задачи, на которые ответ нужно дать в виде числа, как известно, решаются с помощью математических методов. На сегодняшний день существует три основных группы таких методов: аналитические, графические и численные.

При использовании аналитических методов решение задачи удается выразить с помощью формул. Например, если задача состоит в решении простейших алгебраических, тригонометрических, дифференциальных и т.д. уравнений, то использование известных из курса математики приемов сразу приводит к цели.

Преимущество аналитических методов: в результате применения аналитических методов за небольшой отрезок сразу получается точный ответ.

Недостаток аналитических методов: аналитические методы применимы лишь к небольшому числу, как правило, не очень сложных по своей структуре задач. Так, например, до сих пор не удалось решить в общем виде уравнение пятой степени.

Основная идея графических методов состоит в том, что решение находится путем геометрических построений. Например, если уравнение $f(x)=0$ не удается решить аналитически, то строят график функции $y=f(x)$ и абсциссу точки пересечения его с осью *Ox* берут за приближенное значение корня.

Недостаток графических методов: в результате применения графических методов ответ получается с погрешностью, недопустимой в силу своей большой величины.

Основным инструментом для решения сложных математических моделей и задач в настоящее время являются численные методы. Они сводят решение задачи к выполнению конечного числа арифметических действий над числами и дают результат в виде числового значения с погрешностью, приемлемой для данной задачи.

Численные методы разработаны давно. Однако при вычислениях вручную они могли использоваться лишь для решения не слишком трудоемких задач. С появлением компьютеров, которые за короткое время могут выполнить миллиарды операций, начался период бурного развития численных методов и внедрения их в практику.

Это позволяет программистам использовать эти методы для решения разнообразных математических задач, таких как нахождение корней уравнений, вычисление интегралов, аппроксимация функций и многое другое. Знание численных методов является важным навыком для любого программиста, работающего с математическими задачами, и понимание их работы поможет улучшить качество и эффективность программного кода. Они также широко применяются в различных областях науки, техники и финансов, делая их незаменимыми инструментами для решения реальных проблем.

Библиографический список

1. Вержбицкий, В. М. Численные методы. Линейная алгебра и нелинейные уравнения / В. М. Вержбицкий. – М.: Оникс 21 век, 2005. – 636 с. – Текст: непосредственный.

2. Вержбицкий, В. М. Численные методы. Математический анализ и обыкновенные дифференциальные уравнения / В. М. Вержбицкий. – М.: Оникс 21 век, 2005. – 400 с. – Текст: непосредственный.

3. Бахвалов, Н. С. Численные методы / Н. С. Бахвалов, Н. П. Жидков, Г. М. Кобельков – М.: Бином. Лаборатория знаний, 2008. – 432 с. – Текст: непосредственный.

4. Ракитин, В. И. Практическое руководство по методам вычислений / В. И. Ракитин – М.: Высшая школа, 1998. – 374 с. – Текст: непосредственный.